

Exercice 1 : (7 pts= 1+1.5+1.5+3)

Répondre aux questions suivantes :

- a- Quel est l'inconvénient de l'utilisation de la structure Parbegin/parend ?
- b- Quel est l'inconvénient de l'utilisation des événements en tant qu'outil de synchronisation ?
- c- Dans le modèle de plusieurs producteurs et plusieurs consommateurs, pourquoi on a utilisé un sémaphore d'exclusion mutuelle pour chaque famille quant à l'accès au tampon au lieu d'un seul pour les deux familles?
- d- Exprimer un RDV entre trois processus à l'aide de sémaphores.

Exercice n°2 : (5 pts)

Soient deux processus cycliques A et B qui communiquent au moyen d'un tampon T ne pouvant contenir qu'un seul article. Ce tampon est utilisé, de façon alternée, pour la communication dans les deux sens (attention, un seul processus utilise à la fois ce tampon).

- Le comportement du processus A est défini comme suit : Il dépose un article dans le tampon puis attend la réponse de B qu'il prélève du tampon avant de déposer à nouveau un autre article et ainsi de suite...
- Le comportement du processus B est : Lorsque B prélève un article du tampon déposé par A , il dépose sa réponse dans le tampon puis se met en attente d'un autre article de A et ainsi de suite...

- Synchroniser les processus A et B au moyen de sémaphores en donnant une solution optimale.

Exercice n°3 : (8 pts=1+2+5)

On s'intéresse à la gestion d'une classe de ressource à M exemplaires partagées par N processus au moyen des moniteurs classiques. Chaque processus est connu par son identité i comprise entre 1 et N et peut demander k exemplaires de la ressource à la fois. Notons qu'un processus peut effectuer d'autres demandes sans être obligé de libérer les ressources déjà acquises.

- La politique d'allocation est définie comme suit :
- A la demande si le nombre de copies de la ressource demandée est disponible, le processus est satisfait ; dans le cas contraire, les exemplaires déjà acquises de la ressource sont réquisitionnées (enlevées au processus) et le processus sera mis en attente du cumul des exemplaires réquisitionnés et demandés.
- A la libération, la priorité est donnée à ceux qui demandent le plus d'exemplaires de la ressource.

1. Donner la forme générale d'un processus.
2. Donner les structures de données principales nécessaires à cette gestion.
3. Etablir la solution à ce problème.

Bon Courage

Exorcise 1 : (7 pts= 1+1.5+1.5+3)

Answer the following questions :

- a- What is the drawback of using the structure Parbegin/parend?
- b- What is the drawback of using events as a synchronization tool?
- c- In the model of several producers and several consumers, why we have used one mutual exclusion semaphore for each family in order to control access to the buffer instead of a single one for both families?
- d- Express a RDV between three processes using semaphores.

Exorcise n°2 : (5 pts)

Consider two cyclic processes *A* and *B* that communicate using a buffer *T* which can only contain one article. This buffer is used alternately for communication in both directions (note that only one process uses this buffer at a time).

- The behaviour of process *A* is defined as follows: It places an article in the buffer, then waits for *B*'s response which it retrieves from the buffer before placing another article and so on...
- The behaviour of process *B* is: When *B* retrieves an article from the buffer placed by *A*, it places its response in the buffer and then waits for another article from *A* and so on...

- Synchronize processes *A* and *B* using semaphores by giving an optimal solution.

Exorcise n°3 : (8 pts=1+2+5)

We consider the management of a class of resource with *M* copies shared by *N* processes using classical monitors. Each process is known by its identity *i* between 1 and *N* and can request *k* copies of the resource at a time. Note that a process can make other requests without being obliged to release the resources it has already acquired.

The allocation policy is defined as follows:

- At request, if the number of copies of the resource requested is available, the process is satisfied; otherwise, the copies of the resource already acquired are requisitioned (removed from the process) and the process must wait for the sum of requisitioned and requested copies.
- At release, priority is given to those requesting the most copies of the resource.

1. Give the general form of a process.
2. Give the main data structures required for this management.
3. Establish the solution to this problem.

Good Luck

Correction de l'Epreuve de Rattrapage (24/25)

Exercice 1 : (7 pts= 1+1.5+1.5+3)

- a. Elle ne permet pas d'exprimer les graphes de précédences non proprement liés.
- b. Car ils sont très délicats quant à l'expression d'un problème général de synchronisation.
- c. Car on doit permettre l'accès concurrent au tampon aux deux familles au maximum à un processus par famille.
- d. Solution de RDV :

Var j : entier := 0 ; Sémaphore S := 0 ; mutex := 1 ;

Processus $P(i$: entier) ; $i=1, 2, 3$

Début

-

$P(\text{mutex})$;

$j:=j+1$;

Si ($j < 3$) Alors $V(\text{mutex})$ $P(S)$;

Sinon $V(S)$; $V(S)$; $V(\text{mutex})$

Fsi;

-

Fin.

Exercice n°2 : (5 pts)

- Solution

Type Tart= ... ;

Var Semaphore SA := 0 ; SB := 0 ;

T : Tart ; Art, Rep : Tart ;

Processus A();

Debut

Repeter

Produire(Art);

Deposer(T, Art);

$V(SB)$;

$P(SA)$;

Prelever(T, Rep);

Traiter(Rep);

Jusqu'à faux;

Fin.

Processus B();

Debut

Repeter

$P(SB)$;

Prelever(T, Art);

Traiter(Art, Rep);

Deposer(T, Rep);

$V(SA)$;

Jusqu'à faux;

Fin.

Exercice n°3 : (8 pts=1+2+5)

1/ Forme générale d'un processus

Processus $P(i$: entier) ;

-

$M.$ demander (i, k) ; // M est le nom du moniteur.

<Utiliser ressources>

$M.$ liberer (i, m) ;

-

2/Structures de données utilisées :

- Une file explicite f , qui garde trace des demandes des processus en exemplaires de la ressource, dont chaque élément est constitué de deux champs, id et $natt$ de type entier. Les procédures de manipulation de f supposées existantes sont :

$premier(f)$: fonction qui retourne le premier élément de f .

$insérer(f, proc)$; insère un élément $proc$ de format id et $natt$ dans f en respectant l'ordre décroissant des valeurs du champ $natt$.

$extraire(f)$: supprime le premier élément de f .

- vide (f)* : fonction qui retourne *vrai* si la file est vide, *faux* sinon.
- un tableau de conditions *c* de taille *n* telle que *c[i]* permet de bloquer le processus *i*.
 - un tableau d'entiers *nacquit* de taille *n* telle que *nacquit[i]* représente le nombre d'exemplaires de la ressource détenues actuellement par le processus *i*.
 - une variable *nbr* qui représente le nombre d'exemplaires de la ressource dans la réserve.

3/ Solution

M : Moniteur ;

Const *N*=... ; *M*=... ;

Type *T*= enregistrement *id, natt* : entier fin ; - 0,75 pts -

Var *nbr* : entier ; *nacquit* : tableau [1..*N*] de entier :=0 ; *proc* : *T* ;

c : Tableau [1..*N*] de condition ;

<Déclaration de la file *f* et ses procédures d'accès>

Procédure demander (*i, k*: entier) ;

Debut

Si (*nbr* ≤ *k*) - 1,75 pts -

Alors *proc.id* := *i* ; /* *i* : identité du processus exécutant cette primitive */ ;

proc.natt := *nacquit[i]* + *k* ; *nbr* := *nbr* + *nacquit[i]* ; *nacquit[i]* := 0

insérer (*f, proc*) ; *c[i].wait()* ;

Sinon *nbr* := *nbr* - *k* ; *nacquit[i]* := *nacquit[i]* + *k* ;

Fsi

Fin ;

Procédure liberer (*i, m* : entier) ;

Var *j* : entier ;

Debut

nbr := *nbr* + *m* ; *nacquit[i]* := *nacquit[i]* - *m* ; - 2 pts -

Tantque (non vide (*f*)) et (*premier(f).natt* ≤ *nbr*)

Faire

j := *premier(f).id* ; *nacquit[j]* := *premier(f).natt* ; *nbr* := *nbr* - *nacquit[j]* ;

extraire (*f*) ; *c[j].signal* ;

Fait ;

Fin ;

Init var *j* : entier ; Debut *nbr* := *M* ; *f* := ϕ ; Pour *j* := 1 à *N* Faire *nacquit[j]* := 0 Fait Fin

- 0,5 pts -

Fin Moniteur.

Correction of remedial exam (24/25)

Exorcise 1 : (7 pts= 1+1.5+1.5+3)

- a. It cannot be used to express precedence graphs that are not properly linked.
- b. Because they are very delicate when it is a matter of expressing a general synchronization problem.
- c. Because we must allow competing access to the buffer to both families at most one process by family.
- d. Solution of the RDV :

Var j : integer :=0 ; Semaphore S :=0 ; mutex :=1 ;

Processes P (i : integer) ; i=1, 2, 3

Begin

-

P(mutex);

j:=j+1;

If (j<3) Then V(mutex) P(S):

Else V(S) ; V(S) ; V(mutex)

EndIf;

-

End.

Exorcise n°2 : (5 pts)

- Solution

Type Tart= ... ;

Var Semaphore SA :=0 ; SB :=0 ; T : Tart ; Art, Rep : Tart ;

Processes A();

Processes B();

Begin

Begin

Repeat

Repeat

Produce(Art);

P(SB);

Place(T, Art);

Retrieve(T, Art);

V(SB) ;

Trait(Art, Rep);

P(SA) ;

Place(T, Rep);

Retrieve(T, Rep);

V(SA);

Trait(Rep);

Until faux;

Until faux;

End.

End.

Exorcise n°3 : (8 pts=1+2+5)

1/ General form of a processes

Processes P (i : integer) ;

-

M. request (i, k) ; // M is the name of the monitor.

<Use resources>

M. release (i, m) ;

-

2/The used data structures :

- An explicit waiting queue *f*, which keeps track of processes' requests for copies of the resource, where each element consists of two fields, *id* and *natt* of type integer. The handling procedures of *f* supposed already exist are :
 - first (f)* : function which returns the first element of *f*.
 - insert (f, proc)* ; inserts an element *proc* with format *id* and *natt* in *f* by respecting the decreasing order of the *natt* field values.
 - empty (f)* : function which returns *true* if *f* is empty, *false* otherwise
- An array *c* of conditions of length *n* such as *c[i]* allows to blocking the process *i*.

- An array of integers *nacquit* of length *n* such that *nacquit[i]* represents the copies number of the resource currently held by process *i*.
- A variable *nbr* which represents the number of resource's copies in the pool.

3/ Solution

M : Monitor ;

Const N=... ; M=...;

Type T= record id, natt :integer end ; - 0,75 pts -

Var nbr : integer ; nacquit :array [1..N] of integer :=0 ; proc : T ;

c : array [1.;N] of condition ;

<Declaration of the queue f and its access procedures>

Procedure requesting (i, k: integer) ;

Begin

- 1,75 pts -

If (nbr<=k)

Then proc.id :=i ; /* i : identity of the process executing this primitive */ ;

proc.natt := nacquit[i]+k ; nbr :=nbr+ nacquit[i] ; nacquit[i] :=0

insert (f, proc) ; c[i].wait() ;

Else nbr :=nbr-k ; nacquit[i] := nacquit[i]+k ;

EndIf

End ;

Procedure release (i, m : integer) ;

Var j : integer ;

Begin

- 2 pts -

nbr :=nbr+m ; nacquit[i] := nacquit[i]- m;

While (not empty (f)) and (first (f).natt<= nbr)

do

j := first (f).id ; nacquit[j] := first(f).natt ; nbr :=nbr- nacquit[j];

extract (f); c[j].signal ;

EndWhile

End ;

Init var j : integer ; Begin nbr :=M ; f := ϕ ; For j :=1 to N Do nacquit[j] :=0 EndFor End

- 0,5 pts -

End Monitor.